ARMLET: Federated Learning System Multi-Criteria Benchmarking Audit - Technical Report -

Baudouin Naline[a], James Sudlow[a], Artur Vieira Pereira[a], Sara Bouchenak[a]

[a] *INSA Lyon, CNRS, LIRIS Lab, France*

Abstract

ARMLET is an extensible framework for multi-criteria benchmarking and audits in Federated Learning (FL). It is designed to be modular and flexible, so that adding features is meant to be practical and simple. The code of ARMLET is available at: `https://github.com/sara-bouchenak/ARMLET`, and its documentation at `https://sara-bouchenak.github.io/ARMLET/`.

1. Main features

ARMLET can be useful if you are interested in the following features:

- We aim to provide a tool that enables multi-criteria benchmarking in FL. With ARMLET, users can evaluate algorithms (e.g., ML models, FL aggregation approaches) or configurations (e.g., model hyperparameters) based on multiple criteria, such as utility, fairness, cost, or privacy. New components, such as fairness and privacy evaluators and metrics, will be gradually integrated into this framework to extend this functionality.
- ARMLET can be used for performing audit by analyzing the results of past experiments. To do this, users need to specify metrics targets in a configuration file and run the audit tool of ARMLET to generate a personalized report.
- ARMLET's data pipeline allows for better integrated data preprocessing, such as data normalization, data cleaning, or features encoding. As a result, components of the data pipeline could be evaluated during benchmarking or audits.

Email addresses: `baudouin.naline@cnrs.fr` (Baudouin Naline), `james.sudlow@insa-lyon.fr` (James Sudlow), `artur.vieira-pereira@insa-lyon.fr` (Artur Vieira Pereira), `sara.bouchenak@insa-lyon.fr` (Sara Bouchenak)

- Due to its configuration management system, our tool makes it easy to share and relaunch experiments, ensuring robust reproductibility. Furthermore, preconfigured configuration files simplifie the process of running experiments, making ARMLET easy to start with. Our framework is also well equipped for studies needing a large number of experiments. It provides several tools to organize experiment logs and automatically load and plot the metrics.

2. Tutorials

At the end of this technical report, we provide three tutorials for getting started with ARMLET:

- Install ARMLET
- Running experiments
- Understanding the FL mode in ARMLET
- Understanding the data pipeline in ARMLET

3. Acknowledgments

ARMLET is based on Fluke [1] to start from an existing FL framework. It serves as the basics for the FL training and evaluation processes by encompassing many FL aggregation algorithms, implementing communication between the server and clients, and providind several tools for data management.

References

[1] M. Polato, Fluke: Federated learning utility framework for experimentation and research, Future Generation Computer Systems 177 (2026) 108241.

ARMLET Tutorial: Installation

First, clone the **ARMLET** repository:

```
git clone https://github.com/sara-bouchenak/ARMLET.git
```

Setup the environment

1. Install conda for managing the environments.
2. Run the following commands:

```
conda create -n armlet python=3.13.5
conda activate armlet
```

Install ARMLET

Install **armlet** using `pip`:

```
cd ARMLET_DIR
pip install .
```

In the documentation, we use `ARMLET_DIR` to refer to the path of the **ARMLET** project and `PROJECT_DIR` to the path of your current project in which you use **ARMLET** and run the command `armlet`.

Software requirements

armlet were developed with Python 3.13.5.

All dependencies listed in `ARMLET_DIR/requirements.txt` are needed (they are installed in the same time when installing **armlet** with `pip`).

ARMLET Tutorial: Running experiments

ARMLET is based on Hydra for simplifying the launch of experiments and ensuring high reproductibily with config files. In the following, we provide examples that illustrate how to efficiently run experiments with our framework, via simple command-line (A) or through experiment config files (B).

Please look at Hydra documentation for further comprehension about Hydra basics.

A- Run experiment(s) from command-line

Run `armlet` to launch a single experiment with the default configuration values.

```
armlet
```

Config values (e.g., `exp.seed`) or config groups (e.g., `data/dataset`) can be directly overrided from the command line. Note that `+` need to be added before the config values or groups that do not have default values.

```
armlet exp.seed=1 data/dataset=ars +data/loading=default
```

For more information about configuration files, config values, or config groups, see Configuration.

Multiple experiments can also be run sequentially by first adding the option `-m` to the command line and then providing multiple values (separated by a comma) for the config values or the config groups. In this case, all combinations of values to be sweeped will be crossed to launch a batch of experiments.

```
armlet -m exp.seed=1,2,3,4,5 data/dataset=ars,heart
```

B- Run experiment(s) from a YAML experiment config file

1. Create a YAML configuration file in `PROJECT_DIR/configs/experiment/` by copying, pasting, and overriding `ARMLET_DIR/configs/experiment/template.yaml`.

Config files saved during a past experiment (`EXP_PATH/config.yaml`) can notably be copied and pasted to `PROJECT_DIR/configs/experiment/` to reproduce the experiment. In that case, add `# @package _global_` at the beggining of the file just as in the following example.

YAML configuration file example:

```yaml
# @package _global_

data:

  cleaning:
    missing_values:
      _target_: armlet.data.cleaning.missing_values.MissingValuesDataCleaningMethod
```

```yaml
    name: default

  dataset:
    _target_: armlet.data.datasets.load_Adult_dataset
    dataset_name: Adult
    path: ./datasets/Adult/raw_data
    sensitive_attributes: ['age', 'gender', 'race']

  distribution:
    _target_: armlet.data.splitter.ArmletDataSplitter.iid

  others:
    client_split: 0.2
    client_val_split: 0.5
    keep_test: false
    sampling_perc: 1.0
    server_split: 0.0
    server_test: false
    server_test_union: true
    server_val_split: 0.0
    uniform_test: false

  seed: 42

eval:
  _target_: armlet.eval.evaluators.MultiCriteriaBinaryClassEval
  eval_every: 1
  locals: true
  metrics:
    fairness: armlet.eval.metrics.BinaryFairnessMetrics
  post_fit: true
  pre_fit: true
  server: true

exp:
  device: cpu
  inmemory: true
  mode: federation
  seed: 42
  train: true

logger:
  _target_: armlet.utils.log.ArmletLog
  json_log_dir: ${paths.output_dir}

method:
```

```yaml
  _target_: armlet.FL_pipeline.FL_algorithms.ArmletCentralizedFL

  hyperparameters:

    client:
      batch_size: 128
      local_epochs: 10
      loss:
        _target_: torch.nn.BCELoss
      optimizer:
        lr: 0.001
        name: SGD
        weight_decay: 0.01
      scheduler:
        gamma: 1
        name: StepLR
        step_size: 1

    model:
      _target_: armlet.utils.net.LogRegression
      input_size: 99
      num_classes: 1

    server:
      loss:
        _target_: torch.nn.BCELoss
      time_to_accuracy_target: null
      weighted: true

paths:
  data_dir: ./datasets
  log_dir: ./logs
  output_dir: ${hydra:runtime.output_dir}
  root_dir: .

protocol:
  eligible_perc: 1.0
  n_clients: 10
  n_rounds: 150

save: {}
```

Just as command line mode, config values that are specified in the experiment config file override the default configuration. Thus, config values that are not provided in this file will be set to their default values.

For greater efficiency, another way of creating experiment config files is to directly provide specific config groups, such as in the following example:

```yaml
defaults:
  - override /data/distribution: dirichlet.yaml
  - override /data/others: union_clients_test.yaml
  - override /save: null
  - override /eval: all.yaml
```

More examples related to config groups can be found in the experiment folder `ARMLET_DIR/configs/experiment/`.

2. Run `armlet` by specifying the path to the experiment config file (from the `PROJECT_DIR/configs/experiment/` folder) in the argument `+experiment=`.

```
armlet +experiment=template
```

Just as running experiments from command-line, multiple experiments can be run sequentially by first adding the option `-m` to the command line and then specifying multiple values (separated by a comma) for the argument `+experiment=`.

```
armlet -m +experiment=example/Adult/log_regression,example/ARS/svm
```

Sweeping capabilities can also be directly added to the experiment config file. To do that, users need to (a) add the following lines at the end of the experiment config file and (b) provide the desired config values for the sweeping. Do not forget to add the `-m` option in the command line.

```yaml
hydra:
  sweeper:
    params:
      exp.seed: 1,2,3,4,5
```

Finally, running experiment config file can be also combined with the command line way of specifying config values.

```
armlet -m +experiment=template exp.seed=1,2,3,4,5
```

ARMLET Tutorial: FL Mode

March 10, 2026

1 Understanding the FL mode in `armlet`

This tutorial allows users to understand how the **FL mode** of **ARMLET** works (i.e., the default mode that serves to run federated learning experiments). It corresponds to the function that is called when running the command `armlet exp.mode=federation` (or just `armlet` as it is the default mode).

1.1 Prerequisites (environment configuration and installation)

If you have not configured the environment and installed `armlet` yet, you have to:

1. Install conda for managing the environments and run the following commands:

```
conda create -n armlet python=3.13.5
conda activate armlet
```

2. Install **`armlet`** using `pip`:

```
cd ARMLET_DIR
pip install .
```

Then, fill the two following variables to detail the main paths and run the command.

[1]:
```python
import os

ARMLET_DIR = "../"
OUTPUT_DIR = os.path.join(ARMLET_DIR, "outputs", "tutorial", "FL_mode")

if not os.path.exists(OUTPUT_DIR):
    os.makedirs(OUTPUT_DIR)
```

1.2 Prepare the configuration

Before calling the run_federation() main function (in `ARMLET_DIR/armlet/run.py`), **ARMLET** uses the Hydra framework to dynamically load the configurations. For further details about **ARMLET** configuration (such as the configuration groups and values), see Configuration documentation page.

In this tutorial, we directly prepare the configuration object in Python by creating dictionaries and using OmegaConf, the YAML based hierarchical configuration system used in Hydra.

The first config dictionary to create is the `cfg_paths`, which contains the main paths.

[2]:
```
from omegaconf import OmegaConf

cfg_paths = OmegaConf.create({
  "data_dir": os.path.join(ARMLET_DIR, "datasets"),
  "log_dir": os.path.join(ARMLET_DIR, "logs"),
  "output_dir": OUTPUT_DIR,
  "root_dir": ARMLET_DIR,
})
```

Then, we need to provide in `cfg_data` information related to the data pipeline (dataset, data cleaning methods, train, validation, and test set splitting, distribution for the data partitioning, and data seed). With the following configurations, we will: (a) load the Adult dataset; (b) perform an IID data partitioning between X clients (the number of clients will be detailed after); (c) split each client dataset into a training set (80% of the client data) and a test set (20%); (d) concatenate the union of the clients test sets to form the server test set; and (e) clean all data by removing missing values.

[3]:
```
cfg_data = OmegaConf.create({
    "cleaning": {
        "name": "default",
        "missing_values": {"_target_": "armlet.data.cleaning.missing_values.
 ↪MissingValuesDataCleaningMethod"},
    },
    "dataset": {
        "_target_": "armlet.data.datasets.load_Adult_dataset",
        "dataset_name": "Adult",
        "path": os.path.join(cfg_paths["data_dir"], "Adult", "raw_data"),
        "sensitive_attributes": ['age', 'gender', 'race'],
    },
    "distribution": {"_target_": "armlet.data.splitter.ArmletDataSplitter.iid"},
    "others": {
        "client_split": 0.2,
        "client_val_split": 0.0,
        "keep_test": False,
        "sampling_perc": 1.0,
        "server_split": 0.0,
        "server_test": False,
        "server_test_union": True,
        "server_val_split": 0.0,
        "uniform_test": False,
    },
    "seed": 42,
})
```

`cfg_eval` contains the configuration values related to the evaluation. In this tutorial, we use the armlet.eval.evaluators.MultiCriteriaBinaryClassEval evaluation class and evaluate the global model at each round (on the server side with the server test set).

[4]:
```
cfg_eval = OmegaConf.create({
    "_target_": "armlet.eval.evaluators.MultiCriteriaBinaryClassEval",
    "eval_every": 1,
    "locals": False,
    "post_fit": False,
    "pre_fit": False,
    "server": True,
})
```

`cfg_exp` details information about how to run the experiment. Here, we choose the federation mode, enable training, specify cpu as the device, and fix an experiment seed.

[5]:
```
cfg_exp = OmegaConf.create({
    "device": "cpu",
    "inmemory": True,
    "mode": "federation",
    "seed": 42,
    "train": True,
})
```

`cfg_logger` is required to specify the logging class to be used during the experiment. In this tutorial, we use armlet.utils.log.ArmletLog, the minimal logger provided by **ARMLET** (it only saves the results in a JSON file).

[6]:
```
cfg_logger = OmegaConf.create({
    "_target_": "armlet.utils.log.ArmletLog",
    "json_log_dir": cfg_paths["output_dir"],
})
```

`cfg_method` is used to specify the FL algorithm, the ML model, the client hyperparameters, and the server behavior. Here, we use armlet.FL_pipeline.FL_algorithms.ArmletCentralized, i.e., the standard FedAvg algorithm adapted to **ARMLET** pipeline, and consider a logistic regression model. Moreover, at each round, each client performs 10 local epochs by using a SGD optimizer with a learning rate of 0.001 and a weight decay of 0.01.

[7]:
```
cfg_method = OmegaConf.create({
    "_target_": "armlet.FL_pipeline.FL_algorithms.ArmletCentralizedFL",
    "hyperparameters": {
        "client": {
          "batch_size": 128,
          "local_epochs": 10,
          "loss": {"_target_": "torch.nn.BCELoss"},
          "optimizer": {"lr": 0.001, "name": "SGD", "weight_decay": 0.01},
          "scheduler": {"gamma": 1, "name": "StepLR", "step_size": 1},
        },
        "model": {
          "_target_": "armlet.utils.net.LogRegression",
          "input_size": None, # Automatically adjusted after data loading
          "num_classes": None, # Automatically adjusted after data loading
```

```
        },
        "server": {
          "loss": {"_target_": "torch.nn.BCELoss"},
          "time_to_accuracy_target": None,
          "weighted": True,
        },
    },
})
```

`cfg_protocol` details the protocol of the FL process. With these configuration values, we run a FL process with 4 clients, for a total of 2 rounds (as a toy example). During each round, all clients participate to training.

[8]:
```
cfg_protocol = OmegaConf.create({
    "eligible_perc": 1.0,
    "n_clients": 4,
    "n_rounds": 2,
})
```

Finally, we compose each of the previous configuration dictionaries.

[9]:
```
from armlet.utils.configs import ArmletConfiguration

cfg = OmegaConf.create({
    "data": cfg_data,
    "eval": cfg_eval,
    "exp": cfg_exp,
    "logger": cfg_logger,
    "method": cfg_method,
    "paths": cfg_paths,
    "protocol": cfg_protocol,
    "save": {},
})

cfg = ArmletConfiguration(cfg)
```

Now, we are ready to investigate the `run_federation()` function, which is called when `exp.mode=federation`.

1.3 Run the data pipeline

First, **ARMLET** runs the data pipeline to compute the DataSplitter, i.e., the Fluke's Python object that contains all data after processing (launching, data cleaning, train, validation, and test sets splitting, data partitioning, normalization, etc.). As the data pipeline is quite complex, we do not detail it in this tutorial. However, you can find some information about it in the following tutorial: Understanding the **data pipeline** in `armlet`.

[10]:
```
from armlet.data import data_pipeline
```

```
data_splitter, val_data = data_pipeline(cfg)
```

1.4 Adjust the configurations and save them

Once we have prepared data, we setup the Fluke environment and adjust some configuration values (which are related to data, such as the number of classes and the input size of the ML model). We can now save the configuration in a YAML file for allowing reproductibility.

[11]:
```
from fluke import FlukeENV

FlukeENV().configure(cfg)
```

[12]:
```
input_size = data_splitter.data_container.clients_tr[0].tensors[0].shape[-1]
cfg.method.hyperparameters.model.input_size = input_size
if data_splitter.data_container.num_classes <= 2:
    cfg.method.hyperparameters.model.num_classes = 1
else:
    cfg.method.hyperparameters.model.num_classes = data_splitter.data_container.
 ↪num_classes
```

[13]:
```
import yaml

config_path = os.path.join(cfg.paths.output_dir, "config.yaml")
cfg_to_save = cfg.to_dict()
cfg_to_save["paths"]["output_dir"] = "${hydra:runtime.output_dir}"
if "json_log_dir" in cfg_to_save["logger"].keys():
    cfg_to_save["logger"]["json_log_dir"] = "${paths.output_dir}"
yaml.dump(cfg_to_save, open(config_path, "w"))
```

1.5 Instantiate the FL algorithm, evaluator, and logger

Then, we instantiate the FL algorithm we specified in the `cfg_method` dictionary by using Hydra. We do the same for the evaluator and the logger with the `cfg_eval` and `cfg_logger` dictionaries, and pass them to the Fluke environment.

[14]:
```
import hydra

fl_algo = hydra.utils.instantiate(
    cfg.method,
    n_clients=cfg.protocol.n_clients,
    data_splitter=data_splitter,
    val_data=val_data,
    _convert_="all",
    _recursive_=False,
)
```

[15]:
```
evaluator = hydra.utils.instantiate(
    cfg.eval.exclude("locals", "post_fit", "pre_fit", "server"),
```

```
    n_classes=data_splitter.data_container.num_classes,
    sensitive_attributes=cfg.data.dataset.sensitive_attributes,
)
FlukeENV().set_evaluator(evaluator)
```

[16]:
```
log_name = f"{fl_algo.__class__.__name__} [{fl_algo.id}]"
log = hydra.utils.instantiate(cfg.logger, name=log_name)
log.init(**cfg, exp_id=fl_algo.id)
fl_algo.set_callbacks([log])
FlukeENV().set_logger(log)
```

```
                        Configuration
{
    'save': {},
    'data': {
        'cleaning': {
            'name': 'default',
            'missing_values': {
                '_target_':
'armlet.data.cleaning.missing_values.MissingValuesDataCleaningMethod'
            }
        },
        'dataset': {
            '_target_': 'armlet.data.datasets.load_Adult_dataset',
            'dataset_name': 'Adult',
            'path': '../datasets/Adult/raw_data',
            'sensitive_attributes': [
                'age',
                'gender',
```

```
                'race'
            ]
        },
        'distribution': {
            '_target_': 'armlet.data.splitter.ArmletDataSplitter.iid'
        },
        'others': {
            'client_split': 0.2,
            'client_val_split': 0.0,
            'keep_test': False,
            'sampling_perc': 1.0,
            'server_split': 0.0,
            'server_test': False,
            'server_test_union': True,
            'server_val_split': 0.0,
            'uniform_test': False
        },
        'seed': 42
    },
    'eval': {
        '_target_': 'armlet.eval.evaluators.MultiCriteriaBinaryClassEval',
        'eval_every': 1,
        'locals': False,
```

```
        'post_fit': False,
        'pre_fit': False,
        'server': True
    },
    'exp': {
        'device': 'cpu',
        'inmemory': True,
        'mode': 'federation',
        'seed': 42,
        'train': True
    },
    'logger': {
        '_target_': 'armlet.utils.log.ArmletLog',
        'json_log_dir': '../outputs/tutorial/FL_mode'
    },
    'method': {
        '_target_': 'armlet.FL_pipeline.FL_algorithms.ArmletCentralizedFL',
        'hyperparameters': {
            'client': {
                'batch_size': 128,
                'local_epochs': 10,
                'loss': {
                    '_target_': 'torch.nn.BCELoss'
```

```
            },
            'optimizer': {
                'lr': 0.001,
                'name': 'SGD',
                'weight_decay': 0.01
            },
            'scheduler': {
                'gamma': 1,
                'name': 'StepLR',
                'step_size': 1
            }
        },
        'model': {
            '_target_': 'armlet.utils.net.LogRegression',
            'input_size': 99,
            'num_classes': 1
        },
        'server': {
            'loss': {
                '_target_': 'torch.nn.BCELoss'
            },
            'time_to_accuracy_target': None,
            'weighted': True
```

```
            }
        }
    },
    'paths': {
        'data_dir': '../datasets',
        'log_dir': '../logs',
        'output_dir': '../outputs/tutorial/FL_mode',
        'root_dir': '../'
    },
    'protocol': {
        'eligible_perc': 1.0,
        'n_clients': 4,
        'n_rounds': 2
    },
    'exp_id': '3d238d26144244758768939eb4aa6b5c'
}
```

1.6 Run the FL experiment

Finally, we call the `run()` function of the FL algorithm object and close the logger once the FL process is completed. As training progresses, you can follow the evaluation metrics that are computed at the end of each round. Moreover, by closing `ArmletLog`, all metrics are saved in a JSON file (`results.json`) that can be found in the `OUTPUT_DIR` folder (i.e., `ARMLET_DIR/outputs/tutorial/FL_mode` as defined in the beginning of the tutorial).

[17]:
```
try:
    fl_algo.run(cfg.protocol.n_rounds, cfg.protocol.eligible_perc)
except Exception as e:
    log.log(f"Error: {e}")
```

```
    FlukeENV().force_close()
    FlukeENV.clear()
    log.close()
    FlukeENV().close_cache()
    raise e
log.close()
```

```
[UserWarning] /home/bnaline/anaconda3/envs/armlet/lib/python3.13/site-packages/
 ↪rich/live.py:260
install "ipywidgets" for Jupyter support
```

```
                        Round: 1
 {                                                                          ␣
 ↪
     'post-fit': {                                                          ␣
 ↪
         'training_loss': 0.60185,                                          ␣
 ↪
         'support': 4,                                                      ␣
 ↪
         'round': 1                                                         ␣
 ↪
     },                                                                     ␣
 ↪
     'global': {                                                            ␣
 ↪
         'accuracy': 0.78646,                                               ␣
 ↪
         'precision': 0.56932,                                              ␣
 ↪
         'recall': 0.56754,                                                 ␣
 ↪
         'f1': 0.56843,                                                     ␣
 ↪
         'loss': 0.5378,                                                    ␣
 ↪
         'round': 1                                                         ␣
 ↪
     },                                                                     ␣
 ↪
     'comm_cost': 800                                                       ␣
 ↪
 }                                                                          ␣
 ↪
```

```
Memory usage: 960.1 MB [6.11 %]

                        Round: 2
{
    'post-fit': {
        'training_loss': 0.49904,
        'support': 4,
        'round': 2
    },
    'global': {
        'accuracy': 0.81407,
        'precision': 0.65054,
        'recall': 0.53946,
        'f1': 0.58981,
        'loss': 0.4717,
        'round': 2
    },
    'comm_cost': 800
}


Memory usage: 960.1 MB [6.11 %]



                    Overall Performance
{
```

```
    'post-fit': {
        'training_loss': 0.49904,
        'support': 4,
        'round': 2
    },
    'global': {
        'accuracy': 0.81407,
        'precision': 0.65054,
        'recall': 0.53946,
        'f1': 0.58981,
        'loss': 0.4717,
        'round': 2
    },
    'comm_costs': 2000
}
```

ARMLET Tutorial: Data Pipeline

March 10, 2026

1 Understanding the data pipeline in `armlet`

This tutorial details the different steps of the **ARMLET** data pipeline.

In this pipeline, you can: - load the dataset from raw file(s); - load static data (splitted, cleaned, tensors) that were save during previous experiments; - save data at one or multiple specific steps of the data pipeline; - split data across clients; - split data into train, validation, and test sets; - clean data; - perform other preprocessing steps (normalization, encoding, etc.).

1.1 Prerequisites (environment configuration and installation)

If you have not configured the environment and installed `armlet` yet, you can first:

1. Install conda for managing the environments and run the following commands:

```
conda create -n armlet python=3.13.5
conda activate armlet
```

2. Install **`armlet`** using `pip`:

```
cd ARMLET_DIR
pip install .
```

Then, fill the two following variables to detail the main paths and run the command.

[1]:
```
import os

ARMLET_DIR = "../"
OUTPUT_DIR = os.path.join(ARMLET_DIR, "outputs", "tutorial", "data_pipeline")

if not os.path.exists(OUTPUT_DIR):
    os.makedirs(OUTPUT_DIR)
```

1.2 Prepare the configuration

The data pipeline of **ARMLET** requires several configuration values. We define the dictionaries `cfg_paths`, `cfg_data`, `cfg_method`, and `cfg_protocol`, which are explained in details in the tutorial Understanding the **FL mode** in `armlet`.

[2]:
```
from omegaconf import OmegaConf
from armlet.utils.configs import ArmletConfiguration
```

```
cfg_paths = OmegaConf.create({
  "data_dir": os.path.join(ARMLET_DIR, "datasets"),
  "log_dir": os.path.join(ARMLET_DIR, "logs"),
  "output_dir": OUTPUT_DIR,
  "root_dir": ARMLET_DIR,
})

cfg_data = OmegaConf.create({
    "cleaning": {
        "name": "default",
        "missing_values": {"_target_": "armlet.data.cleaning.missing_values.
 ↪MissingValuesDataCleaningMethod"},
    },
    "dataset": {
        "_target_": "armlet.data.datasets.load_Adult_dataset",
        "dataset_name": "Adult",
        "path": os.path.join(cfg_paths["data_dir"], "Adult", "raw_data"),
        "sensitive_attributes": ['age', 'gender', 'race'],
    },
    "distribution": {"_target_": "armlet.data.splitter.ArmletDataSplitter.iid"},
    "others": {
        "client_split": 0.2,
        "client_val_split": 0.0,
        "keep_test": False,
        "sampling_perc": 1.0,
        "server_split": 0.0,
        "server_test": False,
        "server_test_union": True,
        "server_val_split": 0.0,
        "uniform_test": False,
    },
    "seed": 42,
})

cfg_method = OmegaConf.create({
    "_target_": "armlet.FL_pipeline.FL_algorithms.ArmletCentralizedFL",
    "hyperparameters": {
        "client": {
          "batch_size": 128,
          "local_epochs": 10,
          "loss": {"_target_": "torch.nn.BCELoss"},
          "optimizer": {"lr": 0.001, "name": "SGD", "weight_decay": 0.01},
          "scheduler": {"gamma": 1, "name": "StepLR", "step_size": 1},
        },
        "model": {
          "_target_": "armlet.utils.net.LogRegression",
          "input_size": None, # Automatically adjusted after data loading
```

```
            "num_classes": None, # Automatically adjusted after data loading
          },
          "server": {
            "loss": {"_target_": "torch.nn.BCELoss"},
            "time_to_accuracy_target": None,
            "weighted": True,
          },
      },
})

cfg_protocol = OmegaConf.create({
    "eligible_perc": 1.0,
    "n_clients": 4,
    "n_rounds": 2,
})

cfg = OmegaConf.create({
    "data": cfg_data,
    #"eval": cfg_eval,
    #"exp": cfg_exp,
    #"logger": cfg_logger,
    "method": cfg_method,
    "paths": cfg_paths,
    "protocol": cfg_protocol,
    #"save": {},
})

cfg = ArmletConfiguration(cfg)
```

1.3 Data pipeline preparation

In the beginning of the data pipeline, we fix the data seed and compute 3 boolean values that will be used at specific steps: - `is_static_loading` is used to load static data that has been saved during previous experiments; - `is_tensor_data` is used to dynamically load tensors instead of performing the first data processing steps; - `is_saving_mode` is used to save data at different steps of the pipeline.

[3]:
```
import numpy as np
import random

np.random.seed(cfg.data.seed)
random.seed(cfg.data.seed)
```

[4]:
```
is_static_loading = ("loading" in cfg.data.keys()) and ("static" in cfg.data.
 ↪loading.keys()) and cfg.data.loading.static
is_tensor_data = is_static_loading and "tensors" in cfg.data.loading.load_dir.
 ↪split("/")[-1]
```

```
is_saving_mode = "saving" in cfg.data.keys()
```

1.4 Data loading and partitionning

Then, if `is_static_loading` is set to `False`, we perform the default data loading step. It instantiates and runs the dataset loading function specified in `cfg_data["dataset"]`, and splits the data across clients by following the distribution specified in `cfg_data["distribution"]`. When assigning data to clients, the server and the clients also split their data into training, validation, and test sets, according to the configuration values in `cfg_data["others"]`.

If `is_static_loading` is `True`, we load static data that has been saved during previous experiments from the folder specified in `cfg_data["loading"]` and determine if the data is cleaned or not.

Note that, in both cases, the output `splitted_data` is a dictionary that contains the server data (with the keys `server_val` and `server_test`) and the clients data (with the keys `clients_train`, `clients_val` and `clients_test`). At this step, all data are DataFrames, i.e., data structures provided by the Pandas library that contains labeled axes (rows and columns).

In this tutorial, we do not activate the static loading and perform default data loading. Thus, we load the Adult dataset from raw data and split it across 4 clients. Each client has a training set with 80% of its data and a test set with 20%. The server has a test set, which is the union of the clients test sets.

To illustrate this tutorial, we print the training DataFrame of the first client (only the features since the label is separated in a second DataFrame). You can see the different features of the Adult dataset.

[5]:
```
import hydra

from armlet.data.loading import load_data_from_folder
from armlet.data.splitter import ArmletDataSplitter

if is_static_loading:

    splitted_data, is_data_cleaned = load_data_from_folder(cfg.
 ↪to_dict()["data"], cfg.protocol.n_clients)

else:

    data = hydra.utils.instantiate(cfg.data.dataset.exclude("dataset_name"))

    data_splitter = ArmletDataSplitter(
        data_dict=data,
        dist_cfg=cfg.data.distribution,
        **cfg.data.others,
    )

    splitted_data = data_splitter.assign(cfg.protocol.n_clients)
    is_data_cleaned = False
```

```
splitted_data["clients_train"]["client_0"][0].head()
```

[5]:

```
     age workclass  fnlwgt  education education-num      marital-status  \
0  False       NaN   90557       11th             7  Married-civ-spouse
1   True   Private   36228    HS-grad             9            Divorced
2   True   Private  170651  Bachelors            13  Married-civ-spouse
3   True   Private  123430    HS-grad             9  Married-civ-spouse
4  False   Private  181655  Assoc-voc            11  Married-civ-spouse

         occupation   relationship  race  gender  capital-gain  capital-loss  \
0               NaN        Husband  True    True             0             0
1  Transport-moving  Not-in-family  True    True             0             0
2   Exec-managerial           Wife  True   False             0          1977
3   Farming-fishing        Husband  True    True             0             0
4      Adm-clerical        Husband  True    True             0          2377

   hours-per-week native-country
0               8  United-States
1              44  United-States
2              38  United-States
3              65  United-States
4              45  United-States
```

1.5 Data cleaning

After loading the data, we perform data cleaning. For this purpose, we instantiate and run each cleaning methods (e.g., for missing values, outliers, label errors) that we provide in `cfg_data["cleaning"]`. A JSON file (`data_cleaning_metrics.json`) is then saved with the data cleaning metrics. Moreover, depending on the chosen settings in `cfg_data["saving"]`, data can be saved before or after cleaning.

Just as for `splitted_data`, `cleaned_data` is a dictionary that contains the server data and the clients data, but cleaned.

In this tutorial, we only delete missing values without activating the saving mode. After printing the clean training data of the first client, we can see that the first line of the previous DataFrame was deleted as it contains NaN values.

[6]:

```
import json

from armlet.data.saving import save_data
from armlet.data.cleaning import clean_data

if ("cleaning" in cfg.data.keys()) and not is_data_cleaned:

    if is_saving_mode and ("save_data_before_cleaning" in cfg.data.saving.
 ↪keys()) and (cfg.data.saving.save_data_before_cleaning):
```

```
        save_data(splitted_data, cfg.to_dict()["data"], mode="before_cleaning")

    cleaned_data, data_cleaning_metrics = clean_data(splitted_data, cfg=cfg.
↪data.cleaning, sensitive_attributes=cfg.data.dataset.sensitive_attributes)

    data_cleaning_metrics_path = os.path.join(cfg.paths.output_dir,␣
↪"data_cleaning_metrics.json")
    json.dump(data_cleaning_metrics, open(data_cleaning_metrics_path, "w"),␣
↪indent=4)

    if is_saving_mode and ("save_data_after_cleaning" in cfg.data.saving.
↪keys()) and (cfg.data.saving.save_data_after_cleaning):
        save_data(cleaned_data, cfg.to_dict()["data"], mode="after_cleaning",␣
↪metrics=data_cleaning_metrics)

else:
    cleaned_data = splitted_data

cleaned_data["clients_train"]["client_0"][0].head()
```

[6]:

```
     age  workclass  fnlwgt  education education-num      marital-status  \
0   True    Private   36228    HS-grad             9            Divorced
1   True    Private  170651  Bachelors            13  Married-civ-spouse
2   True    Private  123430    HS-grad             9  Married-civ-spouse
3  False    Private  181655  Assoc-voc            11  Married-civ-spouse
4   True  Local-gov   66278    Masters            14       Never-married

         occupation   relationship  race  gender  capital-gain  capital-loss  \
0  Transport-moving  Not-in-family  True    True             0             0
1   Exec-managerial           Wife  True   False             0          1977
2   Farming-fishing        Husband  True    True             0             0
3      Adm-clerical        Husband  True    True             0          2377
4    Prof-specialty  Not-in-family  True   False             0             0

   hours-per-week native-country
0              44  United-States
1              38  United-States
2              65  United-States
3              45  United-States
4              45  United-States
```

1.6 Data preprocessing

The next step is to preprocess all data. In **ARMLET**, we: - perform one hot encoding for the categorical features of each DataFrame; - convert the boolean features and the features of type `category` to numeric (`category` is a special data type we use when loading the dataset to not convert the corresponding feature); - normalize all features using a standard scaler (except for

features tagged as sensitive attributes).

By printing the training DataFrame of the first client, we can see the results of these data preprocessing.

[7]:
```
from armlet.data.processing import one_hot_encoding,␣
↪convert_bool_and_cat_to_num, normalization


one_hot_data = one_hot_encoding(cleaned_data)
numeric_data = convert_bool_and_cat_to_num(one_hot_data)
normalized_data = normalization(numeric_data, cfg.data.dataset.
↪sensitive_attributes)

normalized_data["clients_train"]["client_0"][0].head()
```

[7]:
```
   age    fnlwgt  education-num  race  gender  capital-gain  capital-loss  \
0    1 -1.448676      -0.444587     1       1     -0.147528     -0.219043
1    1 -0.178552       1.128946     1       0     -0.147528      4.665356
2    1 -0.624729      -0.444587     1       1     -0.147528     -0.219043
3    0 -0.074578       0.342179     1       1     -0.147528      5.653600
4    1 -1.164742       1.522329     1       0     -0.147528     -0.219043

   hours-per-week  workclass_Federal-gov  workclass_Local-gov  …  \
0        0.254296              -0.180888            -0.270257  …
1       -0.245010              -0.180888            -0.270257  …
2        2.001870              -0.180888            -0.270257  …
3        0.337514              -0.180888            -0.270257  …
4        0.337514              -0.180888             3.700177  …

   native-country_Portugal  native-country_Puerto-Rico  \
0                -0.037206                   -0.059594
1                -0.037206                   -0.059594
2                -0.037206                   -0.059594
3                -0.037206                   -0.059594
4                -0.037206                   -0.059594

   native-country_Scotland  native-country_South  native-country_Taiwan  \
0                -0.020369             -0.047668              -0.034899
1                -0.020369             -0.047668              -0.034899
2                -0.020369             -0.047668              -0.034899
3                -0.020369             -0.047668              -0.034899
4                -0.020369             -0.047668              -0.034899

   native-country_Thailand  native-country_Trinadad&Tobago  \
0                -0.027332                       -0.025768
1                -0.027332                       -0.025768
2                -0.027332                       -0.025768
```

```
3                -0.027332                       -0.025768
4                -0.027332                       -0.025768

   native-country_United-States  native-country_Vietnam  \
0                      0.308004                 -0.0434
1                      0.308004                 -0.0434
2                      0.308004                 -0.0434
3                      0.308004                 -0.0434
4                      0.308004                 -0.0434

   native-country_Yugoslavia
0                   -0.02467
1                   -0.02467
2                   -0.02467
3                   -0.02467
4                   -0.02467

[5 rows x 99 columns]
```

If the DataFrames have a feature named `image_path`, data is categorized as image. In this case, we load the images based on the column `image_path`, perform some image-specific transformations (e.g., resizing or normalization), and convert the images and labels to tensors. If data is not categorized as image, the dataframes are directly converted to tensors.

We can see in the notebook output that the DataFrame were converted to tensors. Note that the sensitive attributes (specified in the `cfg_data` config file) have been moved to the last columns before tensor conversion so that they can be easily identified during the training/evaluation.

[8]:
```
from armlet.data.processing import convert_dataframes_to_tensors,␣
↪load_and_convert_images_to_tensors


is_image_data = "image_path" in normalized_data["clients_train"]["client_0"][0].
↪columns
if is_image_data:
    tensor_data = load_and_convert_images_to_tensors(normalized_data, cfg.data.
↪dataset)
else:
    tensor_data = convert_dataframes_to_tensors(normalized_data, cfg.data.
↪dataset.sensitive_attributes)

tensor_data["clients_train"]["client_0"][0]
```

[8]:
```
tensor([[-1.4487, -0.4446, -0.1475,  …,  1.0000,  1.0000,  1.0000],
        [-0.1786,  1.1289, -0.1475,  …,  1.0000,  0.0000,  1.0000],
        [-0.6247, -0.4446, -0.1475,  …,  1.0000,  1.0000,  1.0000],
        …,
        [ 0.7367,  1.1289, -0.1475,  …,  1.0000,  0.0000,  1.0000],
```

```
        [ 0.1345,  0.3422, -0.1475,  …,  1.0000,  1.0000,  1.0000],
        [-0.4184, -1.2314, -0.1475,  …,  1.0000,  0.0000,  1.0000]])
```

Depending on the chosen settings in `cfg_data["saving"]`, data can also be saved after conversion to tensors, which can be an interesting choice when dealing with images.

[9]:
```
if is_saving_mode and ("save_data_after_conversion_to_tensors" in cfg.data.
↪saving.keys()) and (cfg.data.saving.save_data_after_conversion_to_tensors):
    save_data(tensor_data, cfg.to_dict()["data"], mode="after_tensors")
```

Note: In the beginning of this tutorial, we compute the `is_tensor_data` boolean based on the configuration values. If this value is set to `True`, all previous steps will be skipped and the tensors will be directly loaded from folder.

```
tensor_data, _ = load_data_from_folder(cfg.to_dict()["data"], cfg.protocol.n_clients)
```

Note: A more flexible way of performing preprocessing is currently under development. It will consist of specifying the desired data preprocessing steps directly in the configuration values, allowing better control and compatibility with other data types.

1.7 Conversion to Fluke data format

After data preprocessing, the final step is to transform the actual data (that is a dictionary of tensors) into the data format required by the FL process of Fluke (i.e., DataSplitter). To do that, we first convert the tensors to FastDataLoader, then transform the dictionary into a DummyDataContainer, and finally pack it in a DummyDataSplitter, which is a simplified version of Fluke's DataSplitter.

[10]:
```
from fluke.data import DummyDataContainer

from armlet.data.processing import convert_tensors_to_fast_data_loaders

num_classes = max([len(val[1].squeeze(1).unique()) for val in␣
↪tensor_data["clients_train"].values()])
final_data = convert_tensors_to_fast_data_loaders(tensor_data, cfg, num_classes)

dummy_data_container = DummyDataContainer(
    final_data["clients_train"],
    final_data["clients_test"],
    final_data["server_test"],
    num_classes,
)
```

[11]:
```
from armlet.data.splitter import DummyDataSplitter

data_splitter = DummyDataSplitter(
    dataset=dummy_data_container,
    distribution="",
```

```
    **cfg.data.others.exclude("client_val_split", "server_test_union",␣
 ↪"server_val_split"),
)

val_data = {k: v for k, v in final_data.items() if k in ["clients_val",␣
 ↪"server_val"]}
```